

Rapport de projet – Médiathèque Django

1. Création d'une application pour une médiathèque

L'objectif de ce projet est de créer une application permettant de moderniser le fonctionnement d'une médiathèque actuellement gérée avec un système papier.

Les différents types de médias proposés par la médiathèque sont :

- les livres ;
- les CD ;
- les DVD ;
- les jeux de plateau.

Tous les médias peuvent être consultés, mais seuls les livres, CD et DVD peuvent être empruntés.

L'application doit être divisée en deux parties :

- une application destinée aux bibliothécaires permettant de gérer les membres, les médias, les emprunts et les retours ;
- une application destinée aux membres permettant de consulter la liste des médias.

Contraintes de gestion de l'application

La gestion des emprunts doit respecter plusieurs règles : un membre peut avoir au maximum trois emprunts simultanés, la durée d'un emprunt est limitée à sept jours et un membre possédant un emprunt en retard ne peut pas effectuer de nouvel emprunt.

2. Analyse et amélioration du code initial

Le code fourni au départ était un programme Python simple contenant les classes `livre`, `dvd`, `cd`, `jeuDePlateau` et `Emprunteur`, ainsi que des fonctions affichant des menus avec `print()`.

Plusieurs problèmes ont été identifiés :

- absence de base de données ;
- répétition d'attributs entre Livre, DVD et CD ;
- absence de véritable gestion des emprunts ;
- absence d'interface web ;
- absence de tests et de journalisation.

En utilisant le framework Django de Python, nous avons pu créer un code plus moderne en réorganisant le projet et en séparant le code dans différents fichiers : `models.py`, `views.py`, `urls.py`, `forms.py` et `tests.py`.

Cette organisation permet de respecter l'architecture MVT (Model-View-Template) utilisée par Django et d'obtenir un code plus lisible et plus facilement maintenable.

3. Modélisation

Afin d'éviter la répétition du code, un modèle parent `Media`, contenant les informations communes comme le titre et la disponibilité, a été créé.

Les modèles suivants héritent de `Media` :

`Media`

├─ Livre

├─ Dvd

└─ Cd

Chaque type possède ensuite une information spécifique : auteur pour un livre, réalisateur pour un DVD et artiste pour un CD.

Le modèle `JeuDePlateau` est volontairement séparé de `Media`, car les jeux sont uniquement consultables et ne doivent pas pouvoir être empruntés.

Les modèles `Membre` et `Emprunt` permettent respectivement de représenter les membres de la médiathèque et les emprunts en cours.

4. Fonctionnalités développées

L'espace bibliothécaire permet de créer, consulter, modifier et supprimer des membres, ainsi que d'ajouter et consulter les différents médias.

La gestion des emprunts respecte les règles demandées :

- maximum de 3 emprunts simultanés par membre ;
- durée maximale de 7 jours ;
- impossibilité d'emprunter lorsqu'un membre possède un emprunt en retard ;
- seuls les livres, DVD et CD sont empruntables ;
- un média emprunté devient indisponible ;
- lors du retour, le média redevient disponible et l'emprunt est supprimé.

L'espace bibliothécaire est protégé par l'authentification Django grâce à `login_required`. La consultation des médias reste accessible publiquement.

Le projet utilise SQLite. L'authentification Django protège l'accès aux fonctionnalités de gestion, mais SQLite ne possède pas d'authentification par utilisateur et mot de passe.

5. Tests et qualité

Des tests automatisés avec `TestCase` ont été réalisés afin de vérifier les principales fonctionnalités comme la gestion des membres, la création des médias, les emprunts, les retours, la limite de trois emprunts et le blocage en cas de retard.

Les tests sont exécutés automatiquement avec la commande :

```
python manage.py test
```

Django utilise une base de données de test indépendante afin de ne pas modifier les données de l'application.

Une fixture `donnees_test.json` permet également de charger rapidement des données de démonstration.

Un système de logging a été ajouté afin d'afficher dans la console les principales opérations réalisées.

6. Conclusion

Ce projet a permis la création d'une application moderne de gestion de médiathèque en se basant sur un programme Python préexistant.

L'utilisation de Django, un framework du langage Python, a permis de créer un code structuré et de mettre en place les différentes fonctionnalités nécessaires au fonctionnement de l'application.

En utilisant les principes de la programmation orientée objet, nous avons notamment mis en application les notions d'héritage, les modèles Django, les vues, les formulaires, les templates, ainsi que l'authentification et les tests automatisés.

L'application finale répond aux principales fonctionnalités demandées pour la gestion de la médiathèque et respecte les règles définies pour le projet.